DG Interns Hub – Cybersecurity Internship

Week 4 Assignment Report:
Network Setup, Apache Web Server, Nmap Scanning & Wireshark Traffic Analysis

Author: Arumugapandi K
Intern ID: DG/SEPTEMBER/CYBER/177
Program: Cybersecurity Internship
Course: B.E. CSE (Cyber Security)
Week: 4

TABLE OF CONTENTS

1. Introduction
2. Week 4 Objectives
3. Network Security Background
4. Lab Environment & Architecture
5. Tools Used
6. Network Configuration
7. Apache Web Server Setup
8. Nmap Scanning Methodology
9. Nmap Results & Analysis
10. Wireshark Traffic Analysis
11. Security Observations
12. Learning Outcomes
13. Limitations
14. Conclusion
15. References

1. INTRODUCTION

In Week 3 I used Nmap and Wireshark against machines on a lab subnet. For Week 4 I went back a step and built the target myself: I checked the network settings of my Kali VM, installed Apache so there was a real service to look at, scanned it with Nmap, and then captured ICMP, HTTP and DNS traffic in Wireshark. The five parts of the task were network setup, service setup, Nmap scanning, Wireshark analysis and findings, and this report follows that order.

Only one virtual machine was available, so the whole practical was done on a single Kali Linux VM running in VirtualBox. Apache, Nmap and Wireshark all ran on that same machine, and the web server was tested through the loopback address `127.0.0.1`. That makes this a scan of the machine against itself, not a scan of one machine from another. I have kept that in view throughout the report and come back to it in Section 13.

Every result in this report comes from the screenshots I took during the practical, the two Nmap output files (`nmap_scan.txt` and `nmap_all_ports.txt`) and the Wireshark capture file (`network_test.pcapng`). Where I add general explanation that is not a lab result, I say so.

2. WEEK 4 OBJECTIVES

Objective	Covered in
Check the basic network configuration of the lab machine (interface, address, route, connectivity)	Section 6
Install and configure a basic web server (Apache)	Section 7
Scan with Nmap and identify open ports, running services and service versions	Sections 8 and 9
Capture and analyse ICMP, HTTP and DNS traffic in Wireshark	Section 10
Present the workflow: network setup → service setup → Nmap scanning → Wireshark analysis → findings	Sections 4, 11 and 14

3. NETWORK SECURITY BACKGROUND

This section is general background, not lab results.

Every network interface on a machine has an IP address. The loopback interface (`lo`, address `127.0.0.1`) is special because traffic sent to it never leaves the machine, which makes it convenient for testing a service

without involving any real network. A normal interface such as `eth0` carries traffic to other machines through a gateway.

A port number tells the operating system which program a packet is meant for. A service that is waiting for connections on a port is said to be listening, and Nmap reports such a port as open. Every open port is a place where the machine accepts input from outside the process, so the list of open ports is a rough measure of how much of the system is reachable. Administrators scan their own systems for the same reason an attacker would: to find out what is exposed.

Scanner output is a conclusion, not a raw record. Wireshark shows the actual packets, so it is the way to check what really crossed the interface. Using both tools together, one to ask questions and one to watch the traffic, is the idea behind this week's workflow.

4. LAB ENVIRONMENT & ARCHITECTURE

The lab is one Kali Linux virtual machine. Apache runs on it as the service under test, and Nmap and Wireshark run on it as the tools. Figure 4.1 shows how the pieces connect. I did not draw a second attacker or target machine because none existed in this practical.

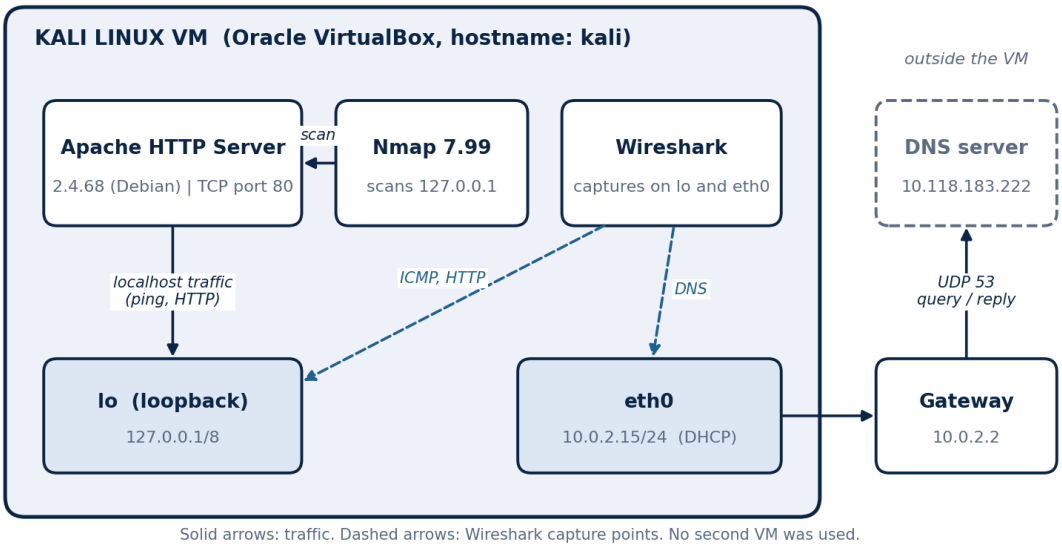


Figure 4.1: Architecture of the single-VM lab used for Week 4

Nmap and the local web requests use the loopback interface. Wireshark was pointed at `lo` for the ICMP and HTTP captures and at `eth0` for the DNS capture. The DNS server in the diagram is drawn outside the VM because it appears in the DNS capture as a separate address (Section 10.3).

Component	Details
Virtualization	Oracle VirtualBox (window title "Kali [Running] – Oracle VirtualBox", Figure 6.1)
Guest OS	Kali Linux, hostname `kali`, user `arumugapandi`
Web server	Apache HTTP Server 2.4.68 (Debian package `2.4.68-2`), TCP port 80
Scanner	Nmap 7.99
Packet analyser	Wireshark, capturing on `lo` and `eth0`

Component	Details
Network mode	Not shown in the screenshots. The `10.0.2.0/24` range with gateway `10.0.2.2` matches VirtualBox's NAT default, so NAT is likely, but I have not confirmed the adapter setting.
Date and time zone	30 September 2026, IST (+0530), as shown in the Nmap and systemd output

5. TOOLS USED

Tool	Purpose	How I used it
Kali Linux	Operating system with the security tools installed	Terminal commands: `ip`, `ping`, `apt`, `systemctl`, `ss`
Apache HTTP Server	Web server	Installed from the Kali repositories and started with `systemctl`; served pages on port 80
Nmap	Port scanning, service detection, default scripts	Default scan, `-sV`, `-sC -sV`, `-oN` and a full `-p-` scan of `127.0.0.1`
Wireshark	Packet capture and display filters	Captures on `lo` (ICMP, HTTP) and `eth0` (DNS) with the filters `icmp`, `http` and `dns`
Firefox, curl	Web clients	Firefox to view the pages (Figures 7.2, 7.4); the captured HTTP request identifies itself as curl (Section 10.2)
VirtualBox	Virtualization	Hosts the Kali VM

6. NETWORK CONFIGURATION

I started by checking what the VM's network looked like before installing anything. `ip addr` lists the interfaces and their addresses, `ip route` shows where traffic is sent, and `ping -c 4 127.0.0.1` sends four echo requests to the loopback address as a basic connectivity test.

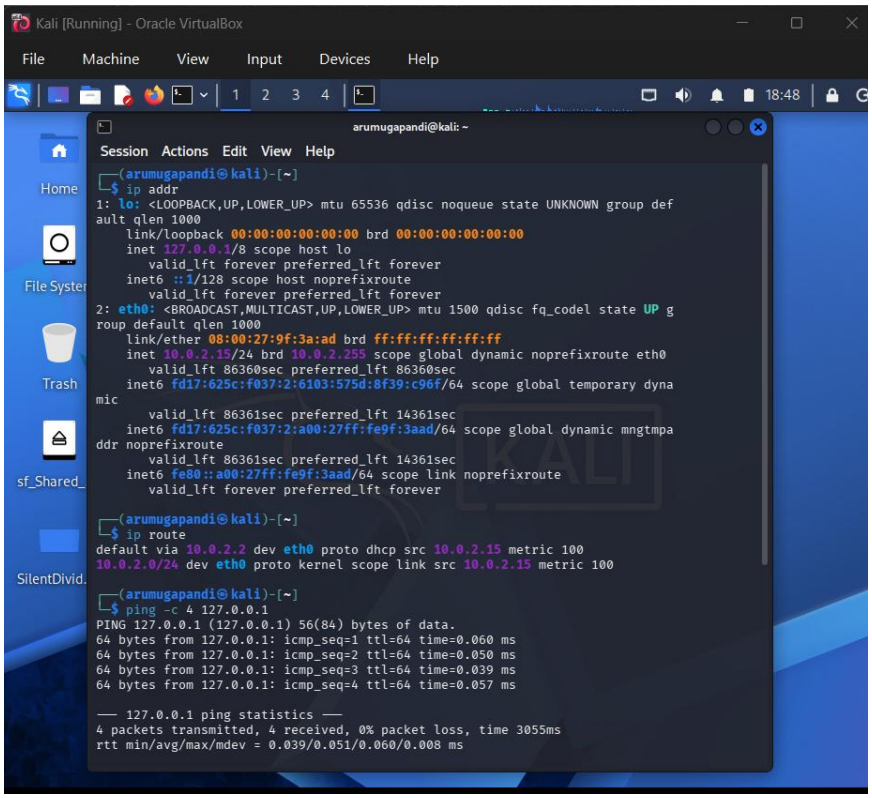


Figure 6.1: ip addr, ip route and ping output on the Kali VM

The screenshot shows two interfaces. `lo` has `127.0.0.1/8`. `eth0` is up with `10.0.2.15/24`, a broadcast address of `10.0.2.255` and the flag `dynamic`, so the address came from DHCP. Its MAC address is `08:00:27:9f:3a:ad`; I use this again in the DNS capture to confirm which interface the packets came from. `eth0` also has IPv6 addresses, including a link-local one starting `fe80::`, but I did not use IPv6 in this practical.

Item	Value in the screenshot	Meaning
Interface address	`10.0.2.15/24` on `eth0`	The VM's address on its virtual network
Subnet	`10.0.2.0/24`	From the connected route, `proto kernel scope link`
Default gateway	`10.0.2.2` via `eth0`	Route learned by DHCP, metric 100
Loopback	`127.0.0.1/8` on `lo`	Used for the Apache, Nmap and ICMP/HTTP tests

The ping test returned four replies out of four sent, 0% packet loss, `ttl=64`, with times between 0.039 ms and 0.060 ms (average 0.051 ms). I want to be clear about what this proves. Pinging `127.0.0.1` shows that the local IP stack is working. It does not test `eth0`, the gateway or anything outside the VM.

7. APACHE WEB SERVER SETUP

7.1 Installing and starting Apache

I installed Apache with `sudo apt update` and `sudo apt install apache2 -y`, then started it with `sudo systemctl start apache2` and checked it with `sudo systemctl status apache2`. Figure 7.1 shows the end of the installation output and the status check.

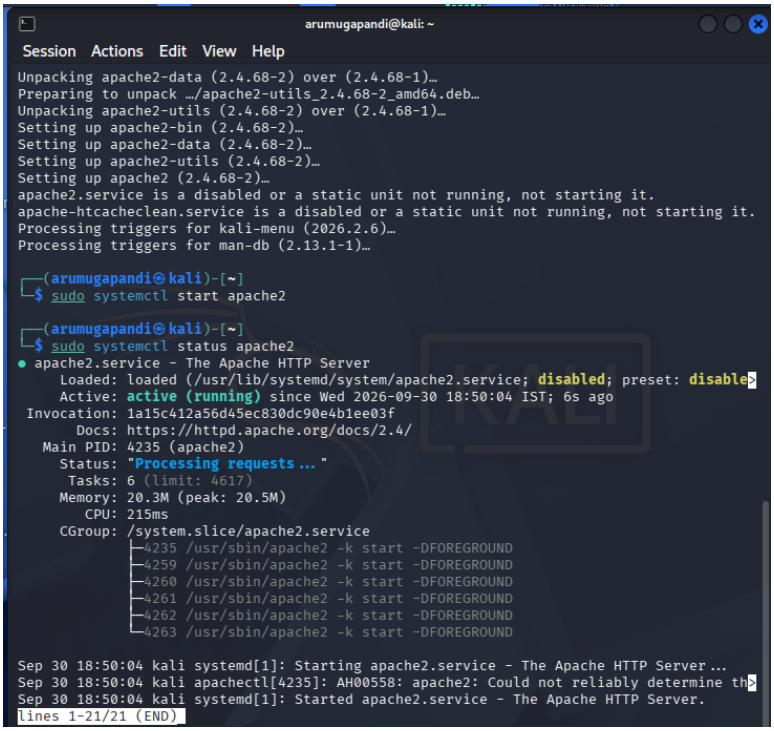


Figure 7.1: Apache installation output and systemctl status

The installation log ends with `apache2 (2.4.68-2)` being set up, followed by a message that the service is disabled or static and is not started automatically. That is why I had to start it myself. The status output then shows `active (running)` since 18:50:04 IST, main PID 4235, and six tasks (the main process plus five workers, PIDs 4259 to 4263). Memory use was 20.3 MB and the status text reads “Processing requests”.

Two smaller details are worth noting. The `Loaded:` line says `disabled`, which means Apache will not start on its own after a reboot unless it is enabled with `systemctl enable`; I did not do that. The journal also shows a line beginning `AH00558`, cut off by the pager in the screenshot. As general background, this is Apache’s usual notice when no `ServerName` is configured, and it did not stop the service from starting.

7.2 Checking the default page

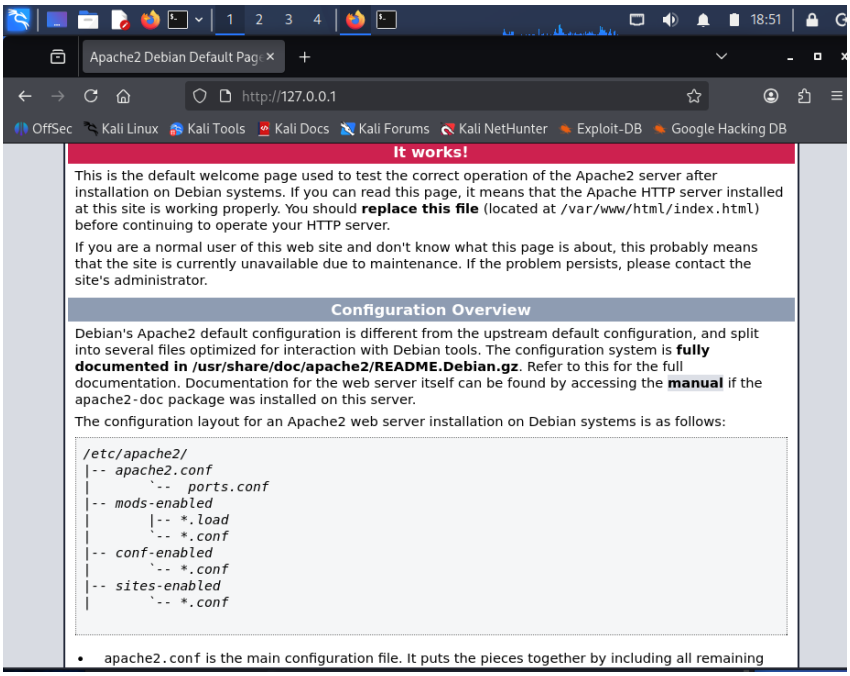


Figure 7.2: Default Apache2 Debian page at http://127.0.0.1

Opening `http://127.0.0.1` in Firefox showed the standard "It works!" page. It confirms that Apache was answering HTTP requests on port 80 and serving files from `/var/www/html`. The page itself says that file should be replaced before the server is used for anything real. It also lists the Debian configuration layout under `/etc/apache2/`, which is a reminder that a default install tells a visitor quite a lot about the software and the operating system behind it.

7.3 Custom page and listening port

To make the server easy to recognise in the scans, I replaced the default page by editing `/var/www/html/index.html` with nano. I then checked which process was holding port 80 with `sudo ss -tulpn | grep :80` (Figure 7.3).



Figure 7.3: Editing index.html and confirming Apache is listening on port 80

The `ss` output shows a single TCP socket in the `LISTEN` state on `*:80`, owned by `apache2`. The `*` means Apache is bound to all local addresses and not only to loopback. Seven `apache2` processes share the socket (PIDs 4235, 4259 to 4263 and 5221). The flags `-tulpn` ask for TCP and UDP listeners, with process names and numeric ports, and the `grep` keeps only the line for port 80.

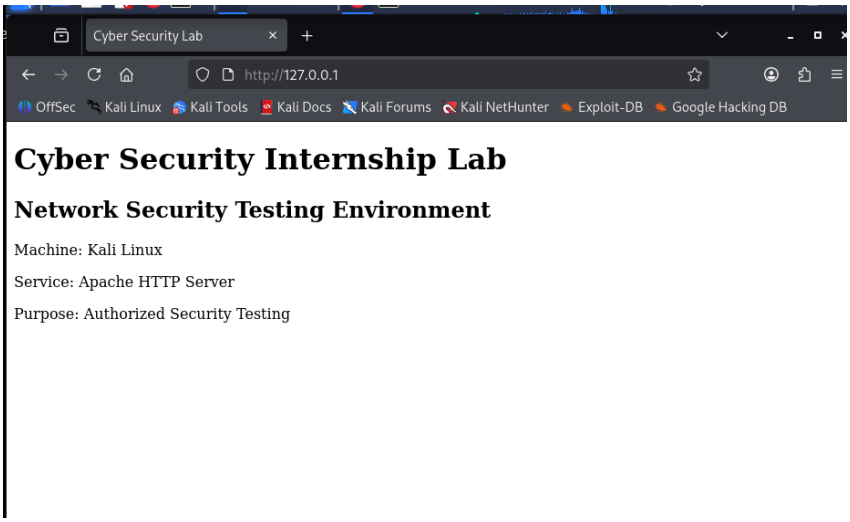


Figure 7.4: Custom test page served by Apache

Figure 7.4 shows the new page with the tab title “Cyber Security Lab”. It states the machine (Kali Linux), the service (Apache HTTP Server) and the purpose (authorized security testing). The same title comes back in the Nmap script output in Section 9, which links the web page to the scan result.

8. NMAP SCANNING METHODOLOGY

Nmap is a network scanner. It sends packets to a target and works out from the replies which ports are open and, with the right options, what is running behind them. Port scanning is useful because it gives a quick, repeatable picture of what a machine exposes. An **open** port has a service accepting connections, a **closed** port has no service but the host still answers (with a TCP reset), and a **filtered** port gives no clear answer, usually because a firewall is dropping the probes. Only open and closed ports appear in my results.

Service detection goes one step further. Instead of guessing the service from the port number, Nmap sends probes to the open port and compares the answers with a database of known services, which is how it can report a product name and version. I ran the scans in the order below, all against `127.0.0.1`.

Command	What it does	Why I ran it
`nmap 127.0.0.1`	Default scan of the 1000 most common TCP ports	First look at what is open
`nmap -sV 127.0.0.1`	`-sV` probes open ports to identify the service and its version	Find out what is behind port 80
`nmap -sC -sV 127.0.0.1`	`-sC` runs Nmap’s default NSE scripts alongside version detection	Collect extra details such as the HTTP title and server header
`nmap -sC -sV 127.0.0.1 -oN nmap_scan.txt`	`-oN` writes the normal-format output to a file	Keep the result as evidence
`nmap -p- -oN nmap_all_ports.txt 127.0.0.1`	`-p-` scans all 65,535 TCP ports	Check that nothing else is listening on a less common port

The command for the last row is taken from the header line inside `nmap_all_ports.txt`, since I did not take a screenshot of that scan.

9. NMAP RESULTS & ANALYSIS

9.1 Default scan

```
┌──(arumugapandi㉿kali)-[~]
└─$ nmap 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 18:58 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

┌──(arumugapandi㉿kali)-[~]
└─$
```

Figure 9.1: Default Nmap scan of 127.0.0.1

The host was up with a latency of 0.000004 s. Nmap reported 999 closed TCP ports (all answered with a reset) and one open port, `80/tcp`, identified as `http`. One open plus 999 closed adds up to the 1000 ports a default scan covers. The scan finished in 0.13 seconds.

9.2 Service and version detection

```
┌──(arumugapandi㉿kali)-[~]
└─$ nmap -sV 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 18:59 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000060s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.68 ((Debian))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.72 seconds
```

Figure 9.2: Service detection with nmap -sV

With `-sV` the result for port 80 became `Apache httpd 2.4.68 ((Debian))`. The scan took 6.72 seconds compared with 0.13 seconds for the default scan, and the difference comes from the extra probes sent to the open port. The version matches the `2.4.68-2` package seen in Figure 7.1, so the installation log and the scanner agree.

9.3 Default scripts and saved output

```
┌──(arumugapandi㉿kali)-[~]
└─$ nmap -sC -sV 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 19:00 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000030s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.68 ((Debian))
|_http-server-header: Apache/2.4.68 (Debian)
|_http-title: Cyber Security Lab

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.89 seconds
```

Figure 9.3: nmap -sC -sV output on the terminal

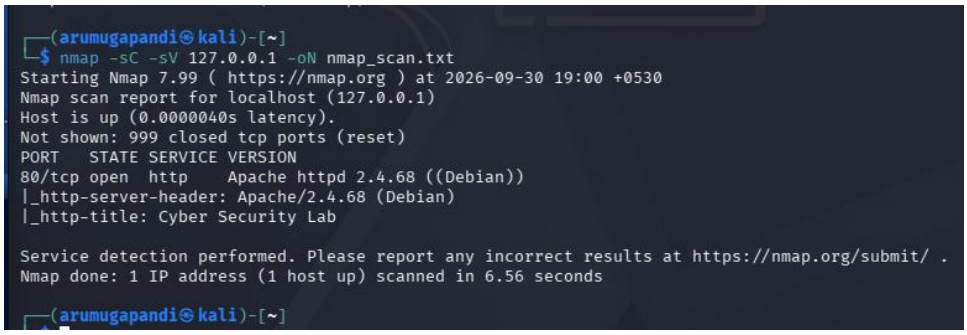


Figure 9.4: nmap -sC -sV with -oN nmap_scan.txt

I ran the combined scan twice, first to the screen (Figure 9.3, 6.89 s) and then again with `-oN` to save it (Figure 9.4, 6.56 s). Both give the same result, and the saved `nmap_scan.txt` matches Figure 9.4 line for line. The two scripts that produced output were `http-server-header`, which returned `Apache/2.4.68 (Debian)`, and `http-title`, which returned "Cyber Security Lab". That title is the one from my custom page in Figure 7.4.

9.4 All-port scan

The default scan only covers 1000 ports, so I repeated the scan across every TCP port. The saved output is shown below.

```
# nmap -p- -oN nmap_all_ports.txt 127.0.0.1
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000010s latency).
Not shown: 65534 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http

# Nmap done at Wed Sep 30 19:01:53 2026 -- 1 IP address (1 host up) scanned in
0.73 seconds
```

Listing 9.1: Contents of nmap_all_ports.txt

Across all 65,535 TCP ports, only port 80 was open. This scan covered TCP only; I did not scan UDP.

9.5 Summary of the findings

Port	State	Service	Version	Interpretation
80/tcp	open	http	Apache httpd 2.4.68 (Debian)	An HTTP service is listening on TCP port 80 and Apache is the software answering requests. It is the only open TCP port on the loopback address.

This is a description of what is exposed, not a vulnerability report. I did not check Apache 2.4.68 against advisories, and I am not claiming that it is vulnerable.

10. WIRESHARK TRAFFIC ANALYSIS

Wireshark records the frames that cross an interface, and display filters narrow the view to one protocol. I used three captures, listed below.

Protocol	Interface	Filter	What the screenshot shows
ICMP	`lo`	`icmp`	8 of 8 packets displayed

Protocol	Interface	Filter	What the screenshot shows
HTTP	`lo`	`http`	2 of 18 packets displayed
DNS	`eth0`	none applied	DNS and ARP frames both visible (the `dns` filter would hide the ARP rows)

10.1 ICMP

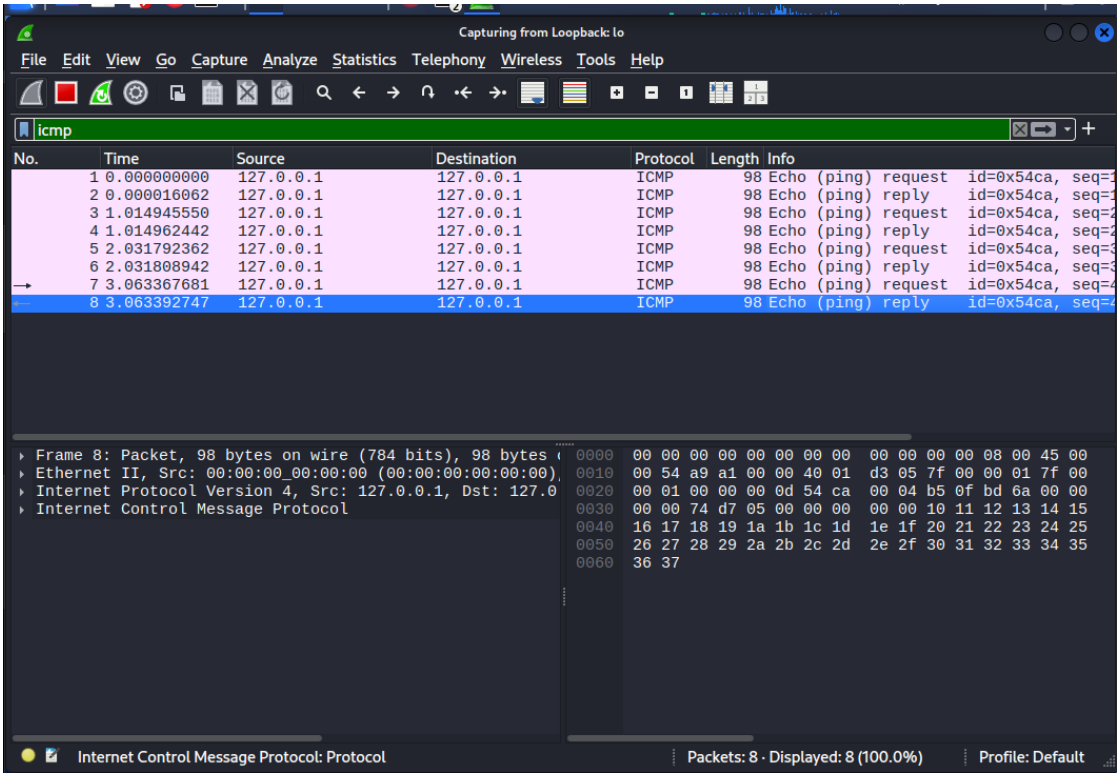


Figure 10.1: ICMP echo requests and replies on the loopback interface

ICMP is the protocol behind `ping`. A host sends an echo request and the receiver answers with an echo reply, which is a simple way to check that an address is reachable. The capture holds eight packets, four request and reply pairs, all between `127.0.0.1` and `127.0.0.1` and all carrying the identifier `0x54ca`. That matches the four packets sent by `ping -c 4` in Figure 6.1.

Each frame is 98 bytes: a 14-byte Ethernet header, a 20-byte IP header, an 8-byte ICMP header and 56 bytes of data, the same 56 data bytes reported by ping. The first reply arrived 16 microseconds after its request, and the requests are about one second apart, which is ping's default interval. Because this is loopback, the Ethernet addresses in the packet details are all zeros.

10.2 HTTP

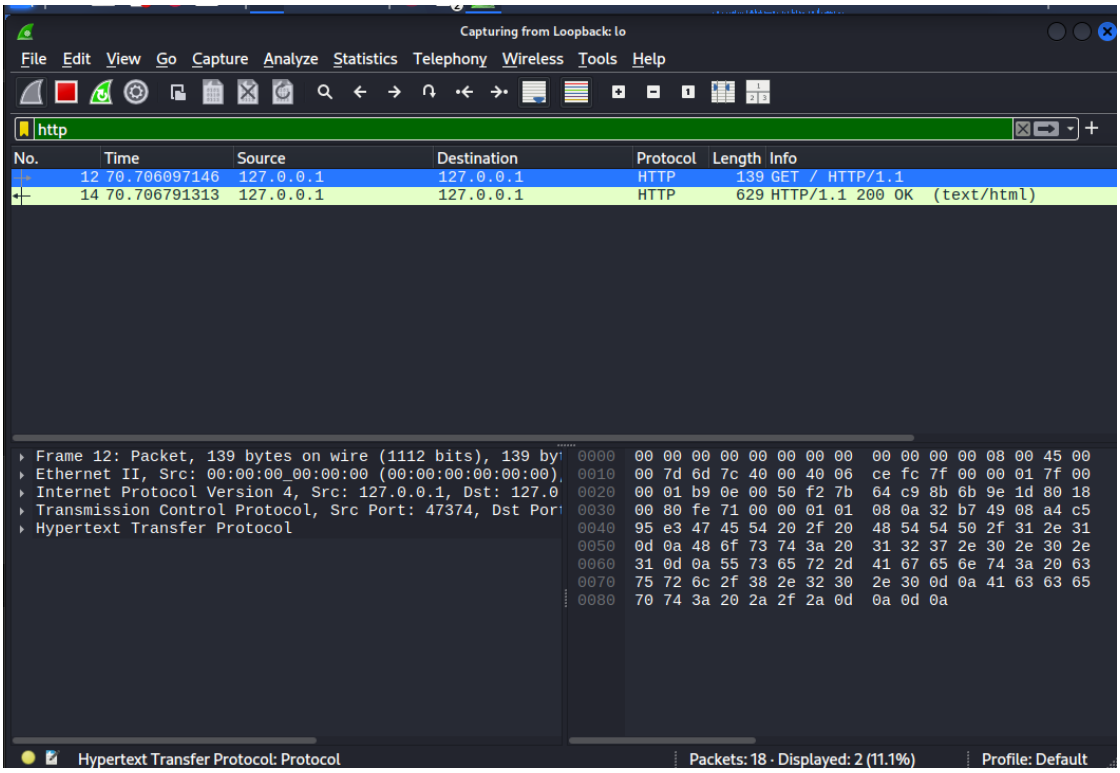


Figure 10.2: HTTP GET request and 200 OK response on the loopback interface

With the `http` filter, two of the 18 captured packets remain. Packet 12 is a `GET / HTTP/1.1` request from `127.0.0.1` port 47374 to `127.0.0.1` port 80 (139 bytes). Packet 14 is the reply, `HTTP/1.1 200 OK (text/html)`, 629 bytes, sent back about 0.7 ms later. The 200 status means Apache found the requested page and returned it, which is the custom page from Figure 7.4.

The bytes pane at the bottom of the screenshot is readable text. It shows the request line, `Host: 127.0.0.1`, `User-Agent: curl/8.20.0` and `Accept: */*`. So this particular request was made with curl and not with Firefox. Nothing in the request is encrypted, which is normal for plain HTTP.

The filter hides the TCP packets around the two HTTP messages. I opened the saved `network_test.pcapng` to see them, and Table 10.1 lists the whole exchange.

No.	Direction	TCP flags	Meaning
9–11	client ↔ server, port 47374 ↔ 80	SYN, SYN/ACK, ACK	Three-way handshake opens the connection
12	client → server	PSH, ACK	HTTP GET request
13	server → client	ACK	Server acknowledges the request
14	server → client	PSH, ACK	HTTP 200 OK response with the page
15–18	both directions	ACK, FIN/ACK	Connection is closed cleanly

Table 10.1: TCP session behind the HTTP request (decoded from network_test.pcapng)

The response headers in the capture file include `Server: Apache/2.4.68 (Debian)`, `Content-Type: text/html` and `Content-Length: 311`. The server header is the same information Nmap reported in Section 9, this time seen directly in a packet.

10.3 DNS

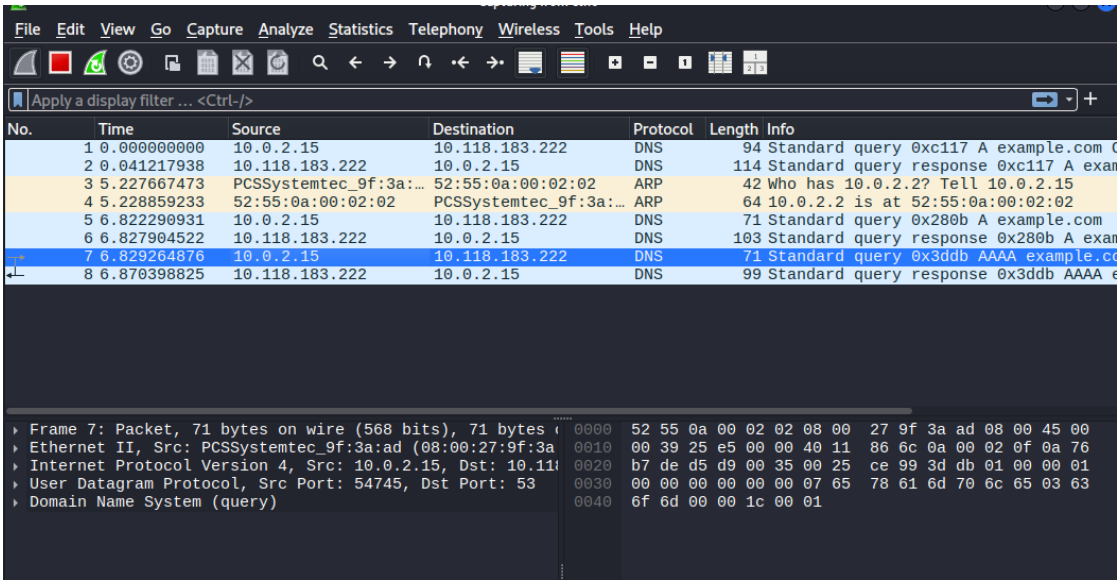


Figure 10.3: DNS queries and responses captured on eth0

DNS turns names into addresses. The capture shows the VM (`10.0.2.15`) sending standard queries for `example.com` to `10.118.183.222` and receiving responses from it. Packet 7, opened in the details pane, is a UDP datagram from source port 54745 to destination port 53, the well-known DNS port, and its type is "query". Queries 0xc117 and 0x280b ask for an `A` record (an IPv4 address) and query 0x3ddb asks for an `AAAA` record (IPv6). Each query is followed by a response with the same transaction ID.

The source MAC in packet 7 is `08:00:27:9f:3a:ad`, the same address as `eth0` in Figure 6.1, which confirms the capture was taken on that interface. The destination MAC in the hex dump is `52:55:0a:00:02:02`. The ARP exchange in packets 3 and 4 ("Who has 10.0.2.2? Tell 10.0.2.15", answered with that MAC) shows this is the gateway's hardware address, so the query was handed to the gateway to reach the DNS server, as expected for an address outside `10.0.2.0/24`.

Response times were about 41 ms for the first query, 5.6 ms for the second `A` query and 41 ms for the `AAAA` query. I did not look into why they differ. What the capture shows is name resolution working, but it cannot tell me which program made the lookups or what the DNS server did with them.

11. SECURITY OBSERVATIONS

I have separated what I saw from what could actually be a risk. An observation is something the evidence shows. A risk needs a situation in which someone could be harmed by it. No vulnerability was found or claimed in this practical.

Observation (from the evidence)	Classification	Comment
Only TCP port 80 is open on `127.0.0.1`, out of all 65,535 ports (Section 9.4)	Observation	A small footprint. Any open service adds attack surface, but this one is intended.
Nmap and the HTTP response both report `Apache/2.4.68 (Debian)`	Possible risk (low)	Version banners help someone match software to known issues. It is information disclosure, not a weakness by itself, and I did not check advisories.

Observation (from the evidence)	Classification	Comment
Apache listens on `*:80`, all local addresses (Figure 7.3)	Observation	My scans only targeted `127.0.0.1`, so I have not tested reachability on `10.0.2.15`. It would matter on a shared network.
HTTP request and response are readable in Wireshark (Figure 10.2)	Possible risk, depends on context	On loopback the traffic stays inside the VM. Over a real network, plain HTTP could be read by anyone on the path. HTTPS avoids this.
DNS queries are unencrypted, and the name `example.com` is visible (Figure 10.3)	Observation	This is how standard DNS on UDP port 53 works. Anyone on the path could see the names being looked up.
The default Apache page shows the software and configuration paths (Figure 7.2)	Possible risk (low)	Minor disclosure on a default install. I replaced the page.
ICMP echo works on loopback (Figure 10.1)	Observation	Useful for connectivity checks. No risk in this setup.
Apache is `disabled` at boot (Figure 7.1)	Observation	Operational detail, not a security issue.

The main practical point is that Nmap and Wireshark answer different questions. Nmap told me what was exposed, and Wireshark showed how the conversation with that service actually looked on the wire. Monitoring at packet level is what lets someone say which traffic is expected and which is not.

12. LEARNING OUTCOMES

- **Linux networking:** reading `ip addr` and `ip route` output, and understanding the difference between loopback and `eth0`.
- **Service management:** installing Apache and using `systemctl` to start it and read its status, including what `disabled` means.
- **Basic Nmap reconnaissance:** default, `-sV`, `-sC`, `-oN` and `-p-` scans, and reading open, closed and version results.
- **Service identification:** matching the version from Nmap to the installed package and to the HTTP response header.
- **Packet capture and filtering:** capturing on `lo` and `eth0` and using the `icmp`, `http` and `dns` filters.
- **Interpreting traffic:** following an ICMP echo pair, a full TCP and HTTP exchange and a DNS query and response.
- **Documentation:** keeping the screenshots, output files and captured data together so that each statement in the report can be traced to evidence.

13. LIMITATIONS

This practical had some limits that affect how far the results can be taken.

- Only one Kali VM was available, so a separate attacker and target topology was not built. Nmap and Wireshark ran on the same machine as Apache.

- The web service was hosted locally and tested through `127.0.0.1`. Loopback traffic does not pass through a real network, so the latency figures (microseconds) and the all-zero Ethernet addresses in Figures 10.1 and 10.2 are not typical of a real network.
- Nmap only scanned the loopback address. I did not scan `10.0.2.15`, so I cannot say what other machines would see. The all-port scan also covered TCP only.
- The work focused on foundational reconnaissance and packet analysis. No exploitation and no vulnerability assessment were performed.
- The screenshots do not show the VirtualBox network mode, the Wireshark version or the command used to generate the DNS lookups.

14. CONCLUSION

In this practical I configured and checked the network settings of a Kali VM, installed Apache and confirmed it was running, replaced its default page, scanned it with Nmap and captured ICMP, HTTP and DNS traffic in Wireshark. Nmap found one open TCP port, 80, running Apache httpd 2.4.68 on Debian, and the all-port scan found nothing else listening on the loopback address.

The Wireshark captures showed the same activity from the packet side: four ICMP echo pairs, a complete HTTP request and 200 response over TCP port 80, and DNS queries and responses over UDP port 53. The main thing I took from the week is how the two tools support each other. The scan gives a result quickly, and the capture lets me confirm what produced it. The results come from a single-VM, loopback-based setup, so the next step would be to repeat the scans between two separate machines.

15. REFERENCES

[1] Nmap Reference Guide – https://nmap.org/book/man.html
[2] Wireshark User's Guide – https://www.wireshark.org/docs/wsug_html_chunked/
[3] Wireshark Display Filter Reference – https://www.wireshark.org/docs/dfref/
[4] Apache HTTP Server Version 2.4 Documentation – https://httpd.apache.org/docs/2.4/
[5] Kali Linux Documentation – https://www.kali.org/docs/
[6] Oracle VirtualBox User Manual – https://www.virtualbox.org/manual/
[7] RFC 792, Internet Control Message Protocol – https://www.rfc-editor.org/rfc/rfc792
[8] RFC 1035, Domain Names – Implementation and Specification – https://www.rfc-editor.org/rfc/rfc1035
[9] RFC 9110, HTTP Semantics – https://www.rfc-editor.org/rfc/rfc9110